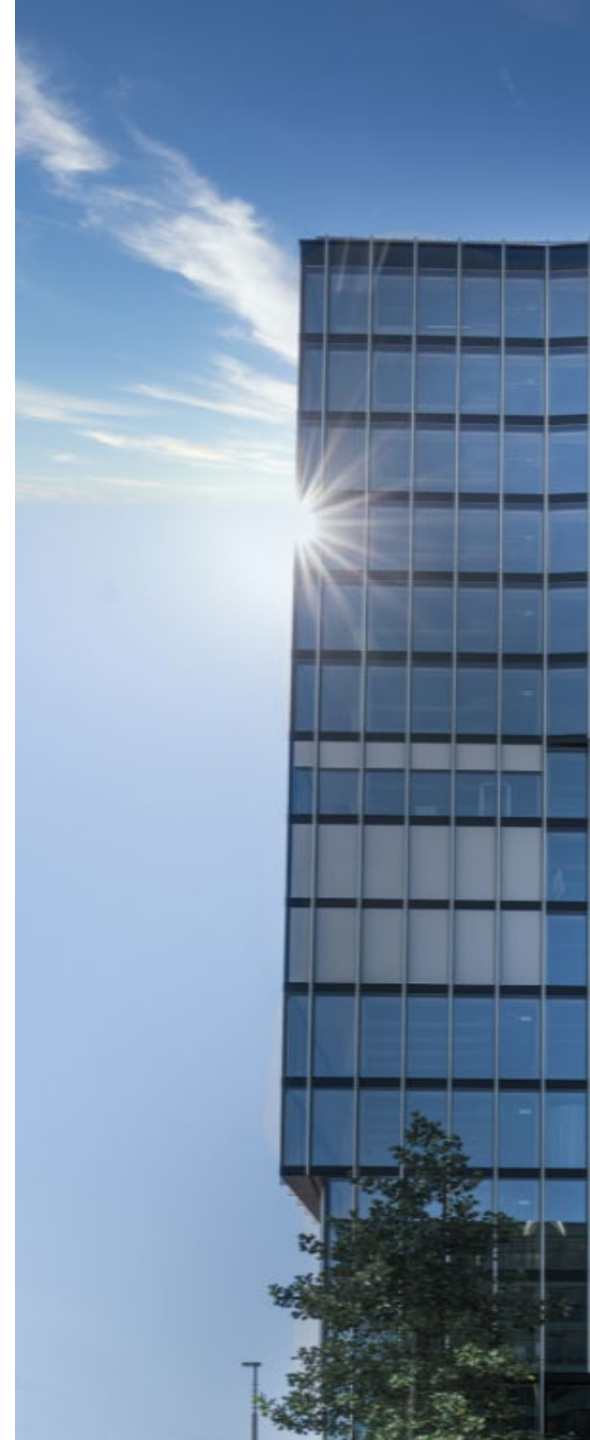


Objektorientierte Programmierung

Exceptionhandling: Testing und Logging

Roland Gisler



Inhalt

- Testen des Exceptionhandlings
- Einsatz von AssertJ
- Logging-Technologien
- Beispiele: Simple Log Facade (SLF), Log4J, BackLog
- Zusammenfassung

Lernziele

- Sie wissen, wie man das Fehlerhandling mit JUnit effizient testen kann.
- Sie kennen und nutzen zusätzliche Libraries beim Testen
- Sie kennen den Nutzen des Einsatzes von Logging.
- Sie können einfaches Logging (z.B. mit SLF4J) nutzen.

Testen des Exceptionhandlings

Testen des Fehlerhandlings

- Weil wir Fehler weniger mögen, «vergessen» wir sehr häufig die Fehlerbehandlung in unseren Programmen seriös zu testen.
- Im Testen des Fehlerhandlings liegt ein **sehr grosses** Potenzial:
Wir vergewissern uns damit, dass Fehlersituationen (die selten eintreten sollten) trotzdem sauber gehandhabt, und diese auch klar als solche signalisiert werden.
- **Super-GAU**: Es tritt ein Fehler auf, aber weil in der Fehlerbehandlung selber ein Fehler passiert, geht dieser Fehler «verloren» → **sehr schwierige** Fehlersuche!
- Aktuelle Testing-Frameworks wie z.B. JUnit bieten uns vielfältige Hilfen an!
 - Nutzen wir diese Möglichkeiten!

JUnit 3 – Exception Testing (**veraltet**)

- **Alte** Technik - aufwändig, aber **leicht** verständlich!

```
public void testExceptionDemoJUnit3() {  
    NumberFormatException save = null;  
    try {  
        new ExceptionDemo().exceptionDemoNonCatch("invalid");  
        fail("Erwartete Exception NICHT aufgetreten!");  
    } catch (NumberFormatException nfe) {  
        save = nfe;  
    }  
    assertNotNull(save);  
    assertEquals("For input string: \"invalid\"", save.getMessage());  
}
```

- Vorgehen:
 - Exception aktiv provozieren (hier mit unerlaubtem Parameter).
 - Exception fangen (**catch**) und danach auswerten.
 - **Wichtig:** Explizites **fail**, wenn Exception nicht eintritt!

JUnit 4 – Exception Testing – Variante 1 (einfach, **veraltet**)

- Sehr einfache Technik über Annotation (ab JUnit 4):

```
@Test(expected = NumberFormatException.class)
public void testExceptionDemoJUnit4Simple() {
    new ExceptionDemo().exceptionDemoNonCatch("invalid");
}
```

- Vorgehen:
 - Per Annotation definieren, welche Exception erwartet wird.
 - Exception aktiv provozieren (hier mit unerlaubtem Parameter).
- Vorteil:
 - **Sehr einfach** zu implementieren und leicht verständlich.
- Nachteil:
 - **Kein Zugriff** auf das Exception-Objekt, um eine inhaltliche Prüfung vorzunehmen, z.B. auf die erwartete Message!

JUnit 4 – Exception Testing – Variante 2 (flexibler, **veraltet**)

- Alternative Technik über Annotationen und «JUnit-Rules»:

```
@Rule
public ExpectedException thrown = ExpectedException.none();

@Test
public void testExceptionDemoJUnit4Rule() {
    thrown.expect(NumberFormatException.class);
    thrown.expectMessage("For input string:");
    new ExceptionDemo().exceptionDemoNonCatch("invalid");
}
```

- Vorgehen:
 - Mit **@Rule** richtet man quasi eine «Falle» ein.
 - Pro Testfall die **ExpectedException** spezifisch konfigurieren.
 - Exception provozieren (hier mit unerlaubtem Parameter).
- Vorteil: **Einfach**, und man kann die Exception **inhaltlich prüfen!**

Aktuell: JUnit 5 – Exception Testing

- Neuste Technik ab JUnit 5 und mit Einsatz von →Lambdas*:

```
@Test
void testExceptionDemoAssertJUnit5() {
    Exception e = assertThrows(NumberFormatException.class, () -> {
        new ExceptionDemo().exceptionDemoNonCatch("invalid");
    });
    assertEquals("For input: \"invalid\"", e.getMessage());
}
```

- Vorgehen:
 - **assertThrows(...)** wird eine →**Lambda**-Expression übergeben, welche die zu testende Exception auslöst.
 - Die Exception erhält man direkt als Rückgabewert und kann diese dann beliebig detailliert überprüfen (optional).
- Vorteil: **Einfach**, und man kann die Exception **inhaltlich prüfen!**

* Input zu Lambdas (Funktionale Programmierung) **folgt später**

Empfehlung – Testen des Exceptionhandlings



- Das Testen der Fehlerbehandlung ist oft eine Herausforderung:
Nehmen Sie diese spannende Herausforderung an!
- Häufig gewinnt man beim Testen zusätzliche Erkenntnisse, man wird sich bewusster welche Situationen wann warum auftreten.
- Diese Erkenntnisse fließen wiederum in Ihre Erfahrungen und somit in Ihren Quellcode ein, der dadurch besser wird.
- Aufwand/Nutzen-Verhältnis dennoch im Auge behalten.

➔ Testen Sie das Exception-Handling unbedingt!

Testing-Library: AssertJ (optional)

AssertJ – «Fluent assertions» für JUnit

- **Fluent-API** (fliessend) ist ein API-Stil, der besser und einfacher verständlich ist, und die Codevervollständigung von IDEs sehr gut unterstützt (haben wir bei EqualsVerifier schon kennengelernt)!
- Klassisches JUnit-Assert mit der grossen Gefahr die Argumente zu vertauschen:

```
- @Test
  public void testQuadrantOne() {
      assertEquals(1, new Point(1, 1).getQuadrant());
  }
```

- Viele statische Methoden, grosse Auswahl, unübersichtlich.

- Alternativ mit **AssertJ** (zusätzliche, ergänzende Library):

```
- @Test
  public void testQuadrantOne() {
      assertThat(new Point(1, 1).getQuadrant()).isEqualTo(1);
  }
```

➔ Methoden für Vergleiche sind **typisiert** und aussagekräftiger.

AssertJ – Testen von Exceptions – Beispiel

- Beispiel:

```
@Test
public void testExceptionHandlingWithAssertJ() {
    assertThatThrownBy(() -> {
        new Demo().exceptionDemoNonCatch("invalid");
    })
        .isInstanceOf(NumberFormatException.class)
        .hasMessage("For input string: \"invalid\"");
}
```

- Nutzt wie JUnit 5 auch → Lambda-Expressions, ist aber bereits seit JUnit 4 nutzbar.
- AssertJ - Quelle: <https://assertj.github.io/doc/>
 - Sehr gute Dokumentation mit vielen Beispielen.
- AssertJ ist im Template bereits integriert und kann genutzt werden!

Demo

Logging-Technologie

Ausgaben auf die Konsole (stdout und stderr)

- Zur Kontrolle unserer Programme haben wir bisher meistens mit `System.out.println(...)` Ausgaben auf die Konsole gemacht.
- Bei vielen Anwendungen ist das eine **wenig sinnvolle Idee**:
 - Konsole ist häufig gar nicht sichtbar.
 - Konsole hat einen beschränkten Ausgabepuffer (Queue!).
 - Ausgaben gehen nach Terminierung des Programmes verloren.
- Bei Serveranwendungen ist es eine **häufig eine sinnlose Idee**:
 - Sehr viele Ausgaben «rauschen» einfach ungesehen durch.
 - Niemand sitzt im Serverraum und guckt zu!
 - Ausnahme: Docker (leichtgewichtige Virtualisierung) – Standard!
- Nebenbei: Auch beim Testen sind `System.out.println(...)` innerhalb von JUnit-Testfällen mehrheitlich **absoluter Nonsense**!
 - ➔ Niemand liest diese (vielen und oft wenig aussagekräftigen) Ausgaben!

Logging mit Logging-Framework

- **Besser:** Einsatz von so genannten **Logging-Frameworks**.
- Sie bieten eine definierte, **einfache Schnittstelle**, damit ein Programm wichtige Informationen «loggen» kann.
 - Typisch unter verschiedenen **Levels** (Filterung möglich):
Error, **Warning**, **Info**, **Debug**, **Trace** etc.
- **Wohin** diese Logs gehen, kann per Laufzeitkonfiguration (ohne Codeänderung und Neukompilation!) festgelegt werden. Beispiele:
 - In **Dateien** schreiben, z.B. mit Log-Rotate nach Tagen separiert.
 - Über das **Netzwerk** an zentralen Logging-Server übermitteln (z.B. **rsyslog**, **Flume**, **logstash**, **Splunk** etc.).
 - Bei verteilten Anwendungen direkt in Clouddienste übermitteln (z.B. **loggly**, **papertrail** etc.).
 - Während der Entwicklung einfach auf die Konsole schreiben.

Verschiedene Logging-Frameworks

- Log4J – Eines der ersten und populärsten Logging-Frameworks.
 - Seit 1999 von Ceki Gülcü (Java 1.2), 2015 deprecated.
- Apache Log4J **2** – Aktuelle Generation von Log4J (Version 2.x).
 - Seit 2012 das wohl populärste Logging-Framework für Java.
 - <https://logging.apache.org/log4j/2.x/>
- LogBack – Designierter Nachfolger von Log4J 2.
 - <https://logback.qos.ch/> - seit 2006, Version 1.0+ seit 2011.
- Java Logging – seit Java 1.4 direkt in Java integriert.
 - seit 2002 – aber sehr selten verwendet.
- Zusätzlich gibt es sogenannte Logging-Fassaden, um Projekte vom verwendeten Logging-Framework unabhängig zu machen.
 - z.B. Apache Commons Logging oder ➔ **SLF4J**.

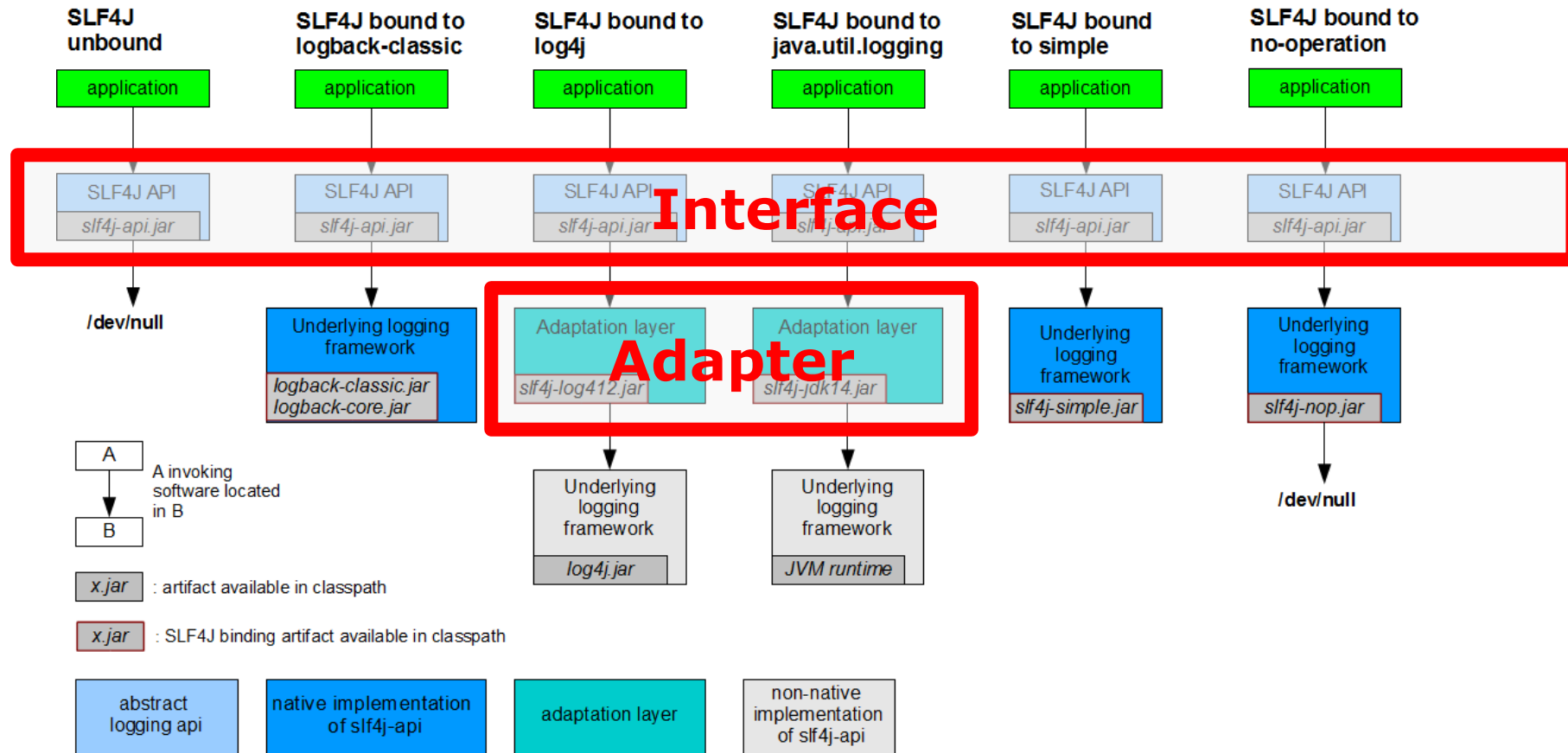
Simple Logging Facade for Java (SLF4J)

Simple Logging Facade for Java (SLF4J)

- Wir nutzen die einheitliche SLF4J-Schnittstelle und loggen dann per Default mit der **LogBack**-Library.
- Begründung: Wenn wir z.B. Libraries oder Komponenten entwickeln, möchten/müssen wir oft verschiedene Logging-Frameworks unterstützen (➔ flexibel, geringer Kopplung).
- Dafür gibt es mit «Simple Logging Facade for Java» (SLF4J), eine «Meta»-Loggingschnittstelle, welche diesen Austausch sehr einfach macht, und die sehr ähnlichen Schnittstellen der verschiedenen Logger vereinheitlicht.
- **LogBack** hat den Vorteil, dass es die SLF4J-Schnittstelle direkt unterstützt (kein Adapter notwendig).

Flexible Alternative: SLF4J – Simple Logging Facade

- Die Architektur und das Design von SLF4J nutzt Komponenten, Schnittstellen, Schichtenbildung und gezieltes Deployment für ein konkretes Einsatzszenario:



SLF4J - Nutzung

SLF4J ist im Template-Projekt als Thirdparty bereits integriert und kann ganz einfach genutzt werden!

1. Import von **LoggerFactory** und **Logger**:

```
import org.slf4j.LoggerFactory;  
import org.slf4j.Logger;
```

2. Pro Klasse eine statische Variable **LOG** für den **Logger**:

```
private static final Logger LOG =  
    LoggerFactory.getLogger(MyClass.class);
```

3. Mit den verschiedenen Levels loggen, was man loggen will:

```
LOG.info("Temperaturwert: {}°C", temperatur);  
LOG.debug("Methode: Ende.");
```

Beispielhafte Loggingausgabe und Konfiguration

- Beispielhafte Ausgabe (exaktes Format und Inhalt konfigurierbar):

```
2023-04-23 15:15:57 DEBUG - catch-Block: Anfang
2023-04-23 15:15:57 ERROR - Konvertierungsfehler aufgetreten!
2023-04-23 15:15:57 DEBUG - catch-Block: Ende
...
2023-04-23 15:15:57 INFO - Temperaturwert: 0.0°C
2023-04-23 15:15:57 DEBUG - Methode: Ende.
```

- Die Konfigurationsdatei(en) sind unter `./src/main/resources` und `./src/test/resources` abgelegt.
 - Meist eine Datei für den «produktiven» und den Test-Code.
 - So kann man z.B. für die Fehlersuche auch in Testfällen loggen.
 - Empfehlung: Per Default deaktiviert, nur bei Bedarf einschalten.
- Die konkrete Konfiguration ist pro Framework unterschiedlich.
 - siehe entsprechende Dokumentation, bei uns ➔ LogBack.

Beispiel: Konfiguration für LogBack

- Einfache Beispielkonfiguration:

```
<configuration scan="false" scanPeriod="30 seconds">
  <appender name="Console" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>%white%d{YYYY-MM-dd HH:mm:ss.SSS} [%thread]
        %highlight(%-5level) %cyan(%logger{20}) - %msg %n</pattern>
    </encoder>
  </appender>
  <root level="INFO">
    <appender-ref ref="Console" />
  </root>
</configuration>
```

- «**appender**» sind Konfigurationen für Logsenken (Konsole, Datei, Netzwerk etc.).
 - Diese Appender lassen sich kombinieren.
 - Mit dem «**level**» lässt sich der Loglevel konfigurieren.
- Der Konfiguration lässt sich auch zur Laufzeit anpassen!

Logging – Einsatz beim Exceptionhandling



- Es ist häufig eine gute Idee, jede Exception im Rahmen ihrer Behandlung **mindestens** zu **loggen**!
 - Adäquater Level: **Error** (oder ggf. **Warning**).
 - Im produktiven Betrieb einer Applikation lässt man z.B. ausschliesslich Logs der Level **Error** und **Warning** persistieren.
 - ➔ Logdatei bleibt im Idealfall leer (weil keine Fehler auftreten).
 - Tritt ein unerwarteter Fehler auf, kann man aufgrund der Logdatei sehr schnell erste nützliche Rückschlüsse ziehen:
 - Wann und wo ist der Fehler aufgetreten, Stacktrace etc.
 - Zur weiteren Fehlersuche kann man den Loglevel dann **temporär** erhöhen, um mehr Kontextinformationen zu erhalten.
- ➔ **Konzept mit welchem Level was geloggt wird notwendig!**

Demo

Zusammenfassung

- Testen des Exceptionhandlings mit JUnit.
 - Historisch verschiedene Techniken / Versionen.
 - Testing des Exceptionshandling sehr empfohlen!
- Logging als ein fundamentales Mittel zur «Behandlung» (Persistierung) von Fehlern.
- Verschiedene Logging-Frameworks.
- Simple Logging Facade (SLF) als flexible Lösung.



Fragen?